

Scan From Both Ends: How One Line Took My Divide and Conquer From TLE to AC

Priyanshu Debnath · priyanshudebnath.me/blog/scan-from-both-ends/

Published 2026-09-26

My solution to Vlad, Misha and Two Arrays was correct and still got TLE on test 14. The fix was to check the split candidates in the order $l, r, l+1, r-1, \dots$ instead of left to right. Here is why that one change turns $O(n^2)$ into $O(n \log n)$.

I got TLE on test 14 with a solution that was completely correct. Then I changed one loop, the order in which I check indices, and it passed comfortably.

Nothing else changed. Same recursion, same math, same checks. Just the order. That trick is worth knowing, because it shows up in far more problems than this one.

1 The problem

E. Vlad, Misha and Two Arrays

2100

Codeforces Round 1102 (Div. 2) · 2 s · 256 MB

divide and conquer, combinatorics, math, dfs and similar

Vlad has a permutation p of length n . For every i he counts the subarrays $[l, r]$ whose minimum is p_i , and writes that count into a_i . You get a and have to count how many permutations produce it, modulo $10^9 + 7$. Here $n \leq 5 \cdot 10^5$ and $a_i \leq 10^{12}$.

2 The idea

Take any segment $[l, r]$ and let k be the position of its minimum. Every subarray inside the segment has exactly one minimum, so the a -values of a valid segment always add up to

$$\sum_{i=l}^r a_i = \frac{m(m+1)}{2}, \quad m = r - l + 1.$$

The minimum splits the segment in two. Everything to its left is a smaller, independent copy of the same problem, and so is everything to its right. That's a Cartesian tree, built from the top.

So how do you find k when all you have is a ? You look for the index where the left part sums to exactly $\binom{i-l+1}{2}$ and the right part sums to exactly $\binom{r-i+1}{2}$. Prefix sums make that an $O(1)$ check per index.

Once you have the split, counting is easy. The left side has $k - l$ values and the right side has $r - k$. The values themselves don't care about positions, only about relative order, so you pick which of the $r - l$ remaining values go to the right subtree:

$$f(l, r) = \binom{r-l}{r-k} \cdot f(l, k-1) \cdot f(k+1, r)$$

If no index passes the check, the answer is 0.

Why only one index can pass the check

Say the real minimum is at k and you're testing some $i < k$. The right part $[i + 1, r]$ already accounts for its $\binom{r-i+1}{2}$ internal subarrays. But the subarray $[i, k]$ crosses the boundary and its minimum sits at k , on the right. That's at least one extra subarray, so the right sum comes out strictly too big. The case $i > k$ is the mirror image, with the left sum too big. So for a valid array the first index that passes *is* the minimum, and you can stop searching there. I also brute-forced every permutation up to $n = 8$ to be sure. The number of distinct valid arrays comes out to the Catalan numbers (1, 2, 5, 14, 42, ...), exactly the number of Cartesian tree shapes. Cute.

3 Where it went wrong

My first version found the split with the obvious loop:

```
1 for (int i = l; i <= r; i++) {
2     if (leftSum(l, i) == tri(i - l) && rightSum(i, r) == tri(r - i)) {
3         // recurse on [l, i-1] and [i+1, r]
4     }
5 }
```

Correct. Also quadratic.

Look at the second sample: $a = 1\ 2\ 3\ 4$, which comes from $p = [4, 3, 2, 1]$. The minimum of every segment is at its **right end**. The loop walks the entire segment to find it, peels off one element, then walks the entire rest again.

$$T(n) = O(n) + T(n - 1) = O(n^2)$$

With $n = 5 \cdot 10^5$ that's around $1.25 \cdot 10^{11}$ steps. Test 14 was almost certainly one of these.

4 The fix

Check the candidates from both ends, alternating: $l, r, l + 1, r - 1, l + 2, \dots$

Both ends (AC)

```
1 for (int step = 0; step < len; step++) {
2     int i = (step % 2 == 0) ? l + step / 2 : r - step / 2;
3     // same check as before
4 }
```

Left to right (TLE)

```
1 for (int i = l; i <= r; i++) {
2     // same check as before
3 }
```

That's it. The whole change.

Here's why it works. If the split lands at distance d from the nearer end, the alternating loop finds it in about $2d$ steps. And d is exactly the size of the **smaller** piece. So a split costs time proportional to the smaller side, never the bigger one:

$$T(n) = O(\min(k, n - k)) + T(k) + T(n - k - 1)$$

That recurrence is $O(n \log n)$, and the proof is a one-liner I really like. Charge the cost of each split to the elements of the smaller piece, one unit each. An element only gets charged when it lands in the smaller half, and each time that happens the segment it lives in at least halves in size. You can't halve n more than $\log_2 n$ times. So each element pays at most $\log n$, and the total is $O(n \log n)$.

The skewed case that killed the first version now costs $O(1)$ per level, since the split is at the right end and the loop checks r second. A perfectly balanced split costs $O(n)$ per level across $\log n$ levels. And everything in between is covered by the charging argument.

If this feels familiar, it should. It's the same reasoning as small-to-large merging (DSU on tree), just applied to searching instead of merging.

5 Numbers

Same code, compiled with `-O2`, input `a = 1 2 ... 500000`:

Scan order	Time
Left to right	over 20 s (I killed it)
Both ends, alternating	0.1 s

Not a constant factor. A different complexity class.

6 The full solution

This is a cleaned-up version of what I submitted, with my template and the segment-tree pruning stripped out since the complexity doesn't need either. Run it on the samples below.

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  using ll = long long;
4  const ll MOD = 1e9 + 7;
5
6  vector<ll> pre, fact, inv;
7
8  ll power(ll b, ll e) {
9      ll r = 1; b %= MOD;
10     for (; e >= 1; b = b * b % MOD) if (e & 1) r = r * b % MOD;
11     return r;
12 }
13 ll C(int n, int k) { return fact[n] * inv[k] % MOD * inv[n - k] % MOD; }
14 ll tri(ll k) { return k * (k + 1) / 2; }
15 ll sum(int l, int r) { return pre[r + 1] - pre[l]; }
16
17 // Number of valid permutations for segment [l, r], 0 if none.
18 ll solve(int l, int r) {
19     if (l > r) return 1;
20     int len = r - l + 1;
21     for (int step = 0; step < len; step++) {
22         int i = (step % 2 == 0) ? l + step / 2 : r - step / 2;
23         if (sum(l, i - 1) == tri(i - l) && sum(i + 1, r) == tri(r - i)) {
24             ll L = solve(l, i - 1);
25             if (!L) return 0;
26             ll R = solve(i + 1, r);

```

```

27         return C(r - 1, r - i) * L % MOD * R % MOD;
28     }
29 }
30 return 0;
31 }
32
33 int main() {
34     ios::sync_with_stdio(false); cin.tie(nullptr);
35     const int MX = 500001;
36     fact.assign(MX, 1); inv.assign(MX, 1);
37     for (int i = 1; i < MX; i++) fact[i] = fact[i - 1] * i % MOD;
38     inv[MX - 1] = power(fact[MX - 1], MOD - 2);
39     for (int i = MX - 1; i > 0; i--) inv[i - 1] = inv[i] * i % MOD;
40
41     int t; cin >> t;
42     while (t--) {
43         int n; cin >> n;
44         pre.assign(n + 1, 0);
45         for (int i = 0; i < n; i++) { ll x; cin >> x; pre[i + 1] = pre[i] + x; }
46         if (pre[n] != tri(n)) { cout << 0 << "\n"; continue; }
47         cout << solve(0, n - 1) << "\n";
48     }
49 }

```

Test cases**sample***Input (truncated):*

```

4
3
1 4 1
4
1 2 3 4
4
1 6 1 2
3

```

Expected output:

```

2
1
3
0

```

single element*Input:*

```

1
1
1

```

Expected output:

```

1

```

balanced tree

Input:

```
1
3
1 4 1
```

Expected output:

```
2
```

skewed, n = 100000

Input: [download \(large file\)](#)

Expected output: [download \(large file\)](#)

Run them in the browser at <https://priyanshudebnath.me/blog/scan-from-both-ends/>.

Stack depth

The recursion can go n levels deep on a skewed input. Codeforces gives you a big stack, so it's fine there. On your own machine you may need `ulimit -s unlimited` before testing a worst case.

The last test case is the skewed worst case at $n = 100000$. It's too big to show inline, so expand it for a preview, or grab everything with **Download all**.

7 Where else this shows up

Any time you recurse by splitting a range at a point you have to *search* for, ask one question: does finding the split cost the size of the whole range, or only the smaller side? If it's the whole range, a skewed input will get you. Searching from both ends is the cheapest way to make it the smaller side.

It's the same pattern behind "find the unique element, split around it" problems, where people usually reach for a sparse table or a stack. Sometimes you don't need either. You just need to stop scanning from the left.

Next time you get a TLE on a divide and conquer you *know* is right, before you rewrite anything, check which way your loop is walking.